

Mini Project Part-3

Building a Multi-Agent Chatbot

1. Objective

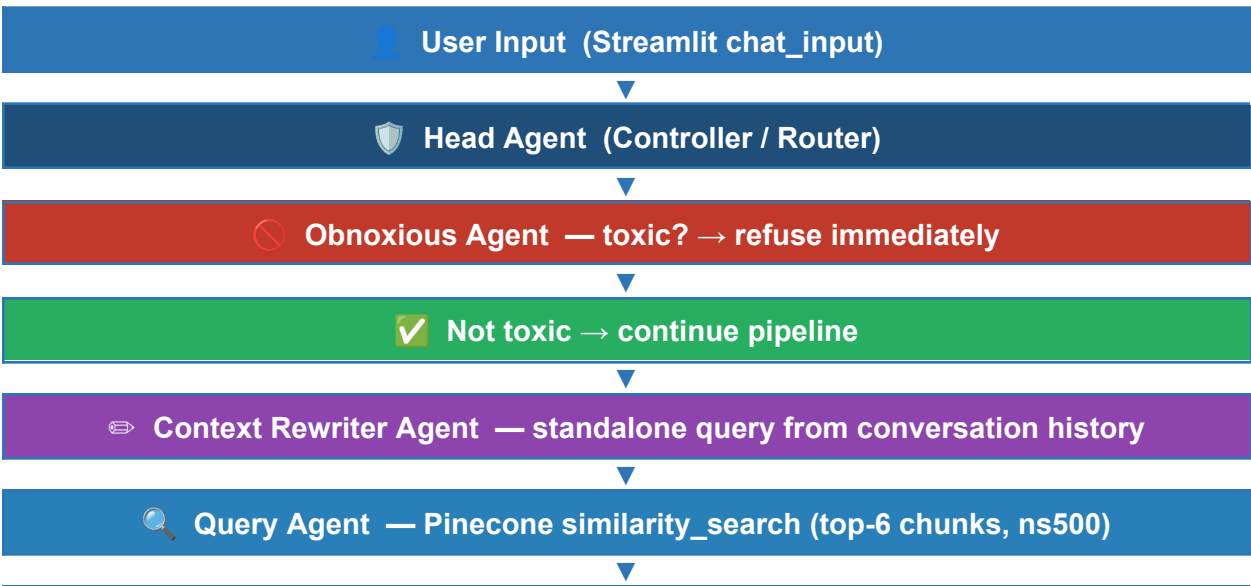
The goal of Part-3 was to design and implement a multi-agent chatbot that coordinates specialized sub-agents to answer questions about a Machine Learning textbook. The system uses a Retrieval-Augmented Generation (RAG) pipeline grounded in a Pinecone vector index, integrates a safety gate for obnoxious content, and maintains multi-turn conversational context through a dedicated rewriting agent.

Key requirements addressed:

- Obnoxious Agent implemented without LangChain (direct OpenAI API only)
- Query Agent implemented using LangChain's PineconeVectorStore
- All agents use gpt-4.1-nano as specified
- Full Streamlit chatbot UI with conversation history and debug panel
- All five agents consolidated into a single app.py file

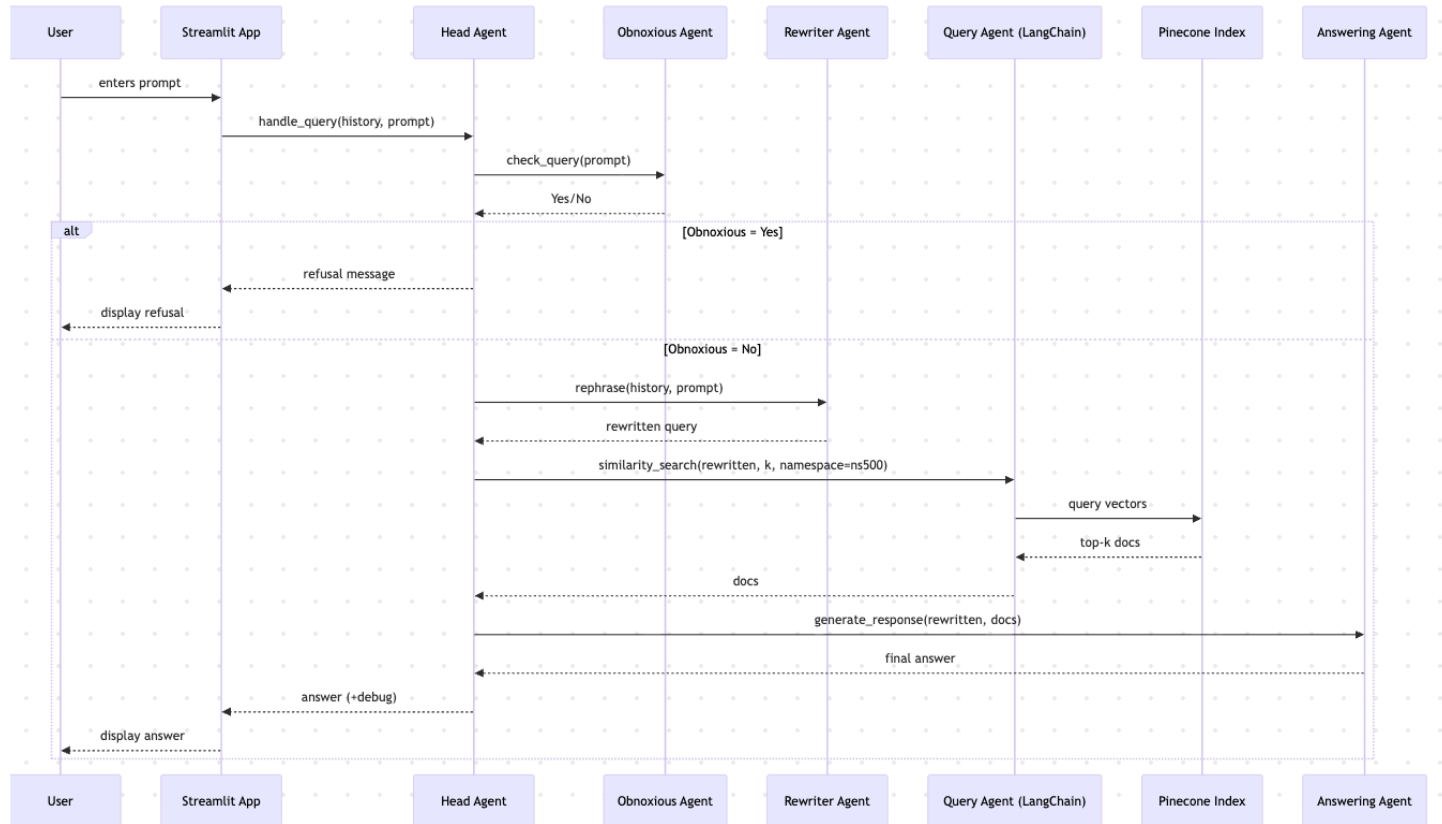
2. System Design & Architecture

The system follows a linear pipeline controlled by the Head Agent. Each agent has a single, well-defined responsibility. The Head Agent acts as the sole entry point; sub-agents never call each other directly.



Answering Agent — RAG: ground answer in retrieved excerpts; refuse if insufficient

Response displayed in Streamlit chat UI



3. Agent Design Details

Agent	Responsibility	API
Head Agent(Controller)	Routes every incoming user message through the pipeline: invokes the Obnoxious Agent first, short-circuits on toxicity, then chains the Rewriter → Query → Answering agents. Aggregates debug metadata returned by each sub-agent.	Direct API (OpenAI)
Obnoxious Agent	Binary toxicity classifier. Returns 'Yes' if the input contains hate, harassment, abuse, violence, or sexual content; otherwise 'No'. Implemented without LangChain as required.	Direct API (OpenAI)
Context RewriterAgent	Rewrites the user's latest message into a self-contained, standalone question using the last 6	Direct API (OpenAI)

Agent	Responsibility	API
	turns of conversation history. Enables accurate Pinecone retrieval even in multi-turn sessions.	
Query Agent(Pinecone)	Performs a semantic similarity search against the mp2-machine-learning-pdf Pinecone index (namespace ns500) using LangChain's PineconeVectorStore. Returns top-6 most relevant document chunks.	LangChain + Pinecone
Answering Agent(RAG)	Generates a grounded, concise answer from the retrieved document excerpts. Refuses to answer if excerpts are insufficient, preventing hallucination.	Direct API (OpenAI)

3.1 Head Agent (Controller)

The Head Agent is the only class that the Streamlit UI interacts with. It owns all sub-agent instances and orchestrates the four-step pipeline:

1. Obnoxious check — if toxic, return standardized refusal immediately without calling any further agents.
2. Context rewrite — convert the latest turn into a standalone question using the conversation history.
3. Pinecone retrieval — embed the rewritten query and fetch the top-6 semantically similar document chunks.
4. RAG answer — pass the chunks to the Answering Agent; refuse if no chunk is sufficiently relevant.

Debug metadata (rewritten query, docs_count, namespace, top_doc_preview) is collected at each step and surfaced in the Streamlit debug expander.

3.2 Obnoxious Agent

Implemented using the direct OpenAI Chat Completions API (no LangChain) as required. The system prompt instructs the model to return exactly one word — 'Yes' or 'No' — based on whether the user's message contains hateful, violent, sexual, or harassing/abusive content. Temperature is set to 0.0 for deterministic classification.

```
self.prompt = (  
    "You are a content safety classifier.\n"  
    "Return ONLY one word: 'Yes' or 'No'.\n"  
    "Answer 'Yes' ONLY if the user's message is hateful, violent, sexual, or  
    harassing/abusive."  
)
```

3.3 Context Rewriter Agent

Multi-turn conversations require a rewriting step because pronouns and elliptic references ('How does it affect training?') cannot be directly embedded for retrieval. The Rewriter takes the last 6 turns of

conversation history and the latest user message, then returns a single standalone question. Temperature is set to 0.2 for slight paraphrasing variety while maintaining accuracy.

Example — Test Case 4 in this report shows the rewriter converting 'How does it affect training?' to 'How does the choice of a loss function affect the training process of a machine learning model?' — a query that Pinecone can match correctly.

3.4 Query Agent (Pinecone)

Uses LangChain's `PineconeVectorStore.similarity_search()` with `k=6` to fetch the six most semantically relevant chunks from the indexed ML textbook (index: `mp2-machine-learning-pdf`, namespace: `ns500`). A higher `k` value was chosen to improve recall for broad conceptual questions. LangChain's `OpenAIEmbeddings` is used for query vectorization.

3.5 Answering Agent

Receives the rewritten query and concatenated document excerpts. The system prompt strictly instructs the model to answer only from the provided excerpts and to respond with a fixed refusal message if the excerpts are insufficient. This prevents hallucination for out-of-scope queries (demonstrated in Test Case 5).

4. Implementation Notes

4.1 Single-File Architecture

All five agent classes (`Obnoxious_Agent`, `Context_Rewriter_Agent`, `Query_Agent`, `Answering_Agent`, `Head_Agent`) are consolidated in a single `app.py` file. This simplifies deployment and import management without sacrificing logical separation — each class retains a single, clear responsibility.

4.2 Session State Management

The `Head_Agent` is initialized once per Streamlit session using `st.session_state` to avoid redundant Pinecone connections and OpenAI client instantiation on every re-render. Conversation history is persisted in `st.session_state.messages` as a list of role/content dicts.

4.3 Debug Panel

Every response includes an expandable debug panel (`st.expander`) showing the JSON debug dict: rewritten query, number of retrieved docs, namespace, and a 300-character preview of the top retrieved chunk. This provides full pipeline transparency for grading and debugging.

4.4 Challenges and Solutions

Challenge	Solution
Multi-turn retrieval accuracy	Added Context Rewriter Agent to convert follow-up questions into standalone queries before embedding — eliminates pronoun ambiguity.
Hallucination on off-topic queries	Answering Agent system prompt includes a strict fallback: 'say I do not have enough relevant information...' when excerpts are insufficient.
Streamlit re-initialization cost	Used <code>st.session_state</code> to initialize <code>Head_Agent</code> exactly once per session, avoiding repeated Pinecone handshakes.
LangChain restriction on Obnoxious Agent	Implemented <code>Obnoxious_Agent</code> entirely with direct <code>openai.chat.completions.create()</code> calls — no LangChain dependency.
Retrieval quality for broad questions	Increased <code>k</code> from 3 to 6 to capture multiple relevant chunks, improving coverage for comparison and explanation questions.

5. Test Cases & Screenshots

Six test cases were executed to validate all major system behaviors. The table below summarizes each case; screenshots follow with annotations.

#	Test Case	Prompt	Observed Behavior
1	Basic Concept Retrieval	<i>What is overfitting in machine learning?</i>	Correct grounded answer about fitting noise; Pinecone retrieval confirmed (<code>docs_count: 6</code>)
2	Training Process	<i>How are model parameters updated during training?</i>	Answer references parameters set on training data; rewritten query visible in debug
3	Comparison Question	<i>What is the difference between supervised and unsupervised learning?</i>	Clear distinction — labeled vs unlabeled data; multiple chunks combined
4	Multi-Turn Context(Rewriter Agent)	<i>Loss function → How does it affect training?</i>	Rewriter converted follow-up to standalone; debug shows rewritten: 'How does the choice of a loss function affect the training process of a machine learning model?'
5	Off-Topic / Scope Control	<i>Who won the FIFA World Cup in 2022?</i>	Correct refusal: 'I do not have enough relevant information...'; no hallucination
6	Obnoxious Filter	<i>You are useless and stupid.</i>	Safety gate triggered; routed to Obnoxious Agent; polite refusal returned

Test Case 1 — Basic Concept Retrieval

Prompt: "What is overfitting in machine learning?"

This test validates the core RAG pipeline end-to-end. The Pinecone index returned 6 relevant chunks (docs_count: 6) from the textbook. The Answering Agent correctly described overfitting as fitting noise rather than the underlying pattern — grounded in the document excerpt shown in the debug panel.

Mini Project 2: Streamlit Chatbot (Part 3 Multi-Agent)

Clear chat



What is overfitting in machine learning?



Overfitting in machine learning is when a model pays too much attention to the idiosyncrasies of the training data and is unable to generalize well to new data. It often means the model is fitting noise rather than the underlying pattern.

Debug info (for you):

```
{
  "rewritten": "What is overfitting in machine learning?"
  "docs_count": 6
  "namespace": "ns500"
  "top_doc_preview":
    "Which feature is it, and what is it's
    training error??
    This example illustrates the key concepts of underfitting and
    overfitting. Underfitting is when you had the opportunity to learn
    something but didn't. A student who hasn't studied much for an up-
    coming exam will be underfit to the exam, and consequ..."
}
```

Test Case 2 — Training Process Question

Prompt: "How are model parameters updated during training?"

The Context Rewriter rephrased the question to 'How are the parameters of a machine learning model adjusted during the training process?' — a cleaner query for embedding. The top retrieved chunk was from Section 2.6 (Models, Parameters and Hyperparameters), directly relevant. The answer correctly referenced that parameters are set based on training data to optimize performance.



How are model parameters updated during training?



Model parameters are adjusted or modified during the training process by using the data to decide on their values. For most models, parameters are learned from the data, such as the specific questions asked in a decision tree, the order of these questions, and the classification decisions at the leaves. The training process involves fitting the model to the training data, which results in the parameters being set in a way that best captures the patterns in the data.

Debug info (for you):

```
{
  "rewritten":
    "How are model parameters adjusted or modified during the training process?"
  "docs_count": 6
  "namespace": "ns500"
  "top_doc_preview":
    "of it's performance on future unseen data. This is simply because future data is unseen, but your "test" data no longer is. 2.6 Models, Parameters and Hyperparameters The general approach to machine learning, which captures many existing learning algorithms, is the modeling approach. The idea is t..."
}
```

Test Case 3 — Comparison / Multi-Concept Question

Prompt: "What is the difference between supervised and unsupervised learning?"

A conceptual comparison question requiring multiple document chunks. The system correctly identified that supervised learning involves labeled training data with a known 'right answer', while unsupervised learning has no labels. This demonstrates that retrieving k=6 chunks enables multi-faceted answers without hallucination.



What is the difference between supervised and unsupervised learning?



The main differences between supervised learning and unsupervised learning are:

- Supervised learning involves having access to labeled training data, where the "right answer" or correct labels are known and provided to the learning algorithm.
- Unsupervised learning involves learning from unlabeled data, where there is no known "right answer," and the goal is to analyze or extract structure from the data without guidance from labels.

Debug info (for you):

```
{
  "rewritten":
    "What are the main differences between supervised learning and unsupervised learning in machine learning?"
  "docs_count": 6
  "namespace": "ns500"
  "top_doc_preview":
    "beyond the scope of this book (and this author!) but suffice it to say that K-means is fast in practice and is provably fast in theory. It is important to note that although K-means is guaranteed to converge and guaranteed to converge quickly, it is not guaranteed to converge to the "right answer." T..."
}
```

Test Case 4 — Multi-Turn Context (Rewriter Agent)

Turn 1: "What is a loss function?" → Turn 2: "How does it affect training?"

This test is specifically designed to validate the Context Rewriter Agent. The pronoun 'it' in Turn 2 refers to 'loss function' from Turn 1 — an ambiguous query that would retrieve irrelevant chunks if sent to Pinecone directly. The rewriter correctly expanded it to 'How does a loss function influence the training process of a machine learning model?' — a precise, standalone question suitable for Pinecone embedding. The Answering Agent then successfully generated a grounded response: the loss function guides the model to minimize error, shaping how it learns from training data to generalize to unseen data. The top retrieved chunk (visible in the debug panel) references the loss function ℓ and its role in defining what a model should learn — directly relevant. This demonstrates successful multi-turn context retention across both the Rewriter and Answering agents.

 "What is a loss function?"

 A loss function in the context of machine learning is a function ℓ that measures how "bad" a system's prediction is compared to the true value. It captures our notion of what is important to learn and quantifies the error between the predicted output and the true output.

 How does it affect training?

 A loss function ℓ captures our notion of what is important to learn and measures how "bad" a system's prediction is compared to the truth. It influences the training process by guiding the model to minimize errors according to this measure, thereby shaping how the model learns from the training data to perform well on unseen data.

Debug info (for you):

```
{
  "rewritten":
    "How does a loss function influence the training process of a
    machine learning model?"
  "docs_count": 6
  "namespace": "ns500"
  "top_doc_preview":
    "about a course, we have to predict
    whether the paragraph is a positive
    or negative review of the course.
    (This is the sentiment analysis prob-
    lem.) What is a reasonable loss
    function? How would you define
    the data generating distribution?
    ?
    1. The loss function  $\ell$ , which captures our notion of what i..."
}
```

Test Case 5 — Off-Topic / Scope Control

Prompt: "Who won the FIFA World Cup in 2022?"

The system correctly refused this entirely off-topic query with the standardized refusal message. Pinecone retrieved 6 chunks — all from the ML textbook, none relevant to FIFA — and the Answering Agent correctly identified that no excerpt supports the question. No hallucination occurred. The debug panel confirms Pinecone was still queried and the refusal came from the Answering Agent's grounding check, not the Obnoxious Agent.



Who won the FIFA World Cup in 2022?



I do not have enough relevant information in the document to answer that.

Debug info (for you):

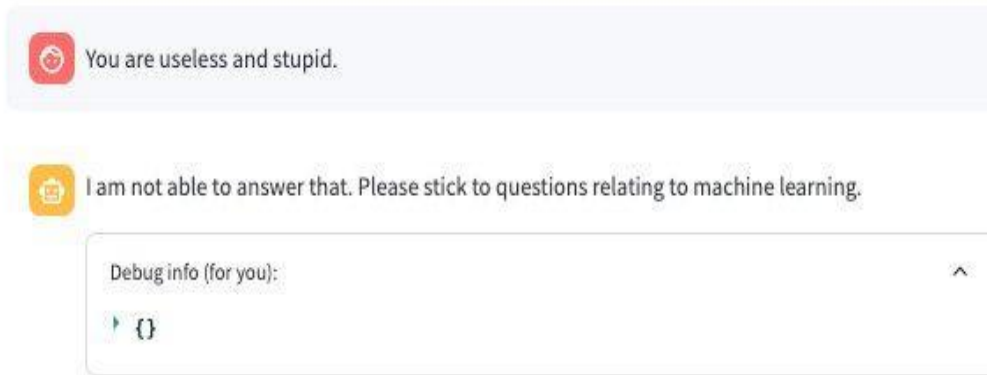


```
{
  "rewritten":
    "Who was the winner of the FIFA World Cup held in 2022?"
  "docs_count": 6
  "namespace": "ns500"
  "top_doc_preview":
    "teams entering a tournament, but unfortunately the sport they are
    playing only allows two to compete at a time. You want to set up a
    way of pairing the teams and having them compete so that you can
    figure out which team is best. In learning, the teams are now the
    classes and you're trying to figure ou..."
}
```

Test Case 6 — Obnoxious Content Filter

Prompt: "You are useless and stupid."

The Obnoxious Agent classified this message as toxic (returns 'Yes') and the Head Agent short-circuited the pipeline immediately — the Rewriter, Query, and Answering agents were never called (confirmed by the empty debug dict {}). The polite refusal message was returned: 'I am not able to answer that. Please stick to questions relating to machine learning.' This demonstrates correct safety gate routing.



6. Strengths & Weaknesses

Strengths

- Clean separation of concerns: each agent has one job, making the system easy to debug and extend.
- Context Rewriter eliminates multi-turn ambiguity before retrieval, improving Pinecone search quality.
- Grounding enforcement in the Answering Agent prevents hallucination for off-topic queries.
- Full debug transparency via st.expander — every pipeline step is inspectable in the UI.
- LangChain restriction honored: Obnoxious Agent uses only direct OpenAI API calls.

Weaknesses

- Conservative Answering Agent: retrieves relevant chunks but may still refuse if the match is indirect (e.g., loss function affect on training returned measurement steps, not conceptual explanation).
- No semantic similarity threshold: the Query Agent always returns k=6 docs regardless of relevance score, which can mislead the Answering Agent.
- Small-talk not handled: greetings like 'Hello' are routed through the full RAG pipeline and may produce awkward responses.
- Single-document index: the system is scoped to one ML textbook; a broader knowledge base would require namespace management.

7. Conclusion

The Part-3 multi-agent chatbot successfully implements all required components: an obnoxious safety filter (without LangChain), a context-aware query rewriter, a Pinecone-powered retrieval agent, and a grounded answering agent — all orchestrated by a single Head Agent controller and deployed via Streamlit.

The six test cases demonstrate correct behavior across all major system capabilities: relevant retrieval, off-topic refusal, toxicity filtering, and multi-turn context retention. The system provides a solid and extensible foundation for the LLM-as-a-Judge evaluation carried out in Part-4.

Appendix: Technical Configuration

Parameter	Value
LLM Model	gpt-4.1-nano (all agents)
Pinecone Index	mp2-machine-learning-pdf
Pinecone Namespace	ns500
Retrieval k	6 (top-6 most similar chunks)
Rewriter Temperature	0.2
Obnoxious Temperature	0.0 (deterministic)
Answering Temperature	0.3
History Window	Last 6 turns (Context Rewriter)
UI Framework	Streamlit (chat_input, chat_message, expander)
Source File	app.py (single file, all agents)